

Module 5: Code Optimization & Target Generation

Basic Blocks, CFG, DAG Optimizations, Register Allocation & Peephole

Module V: Code Optimization & Target Generation — Topics Covered

1. Basic Blocks & Leader Instruction Algorithm
2. Control Flow Graph (CFG) & Loop Dominators
3. DAG Construction & Local Optimizations
4. Machine-Independent Global Optimizations
5. Register Allocation (Graph Coloring Heuristics)
6. Peephole Optimization Techniques

1. Basic Blocks & Leader Identification Algorithm

A **Basic Block** is a sequence of consecutive Three-Address Code statements in which flow of control enters at the beginning and leaves at the end without halt or possibility of branching except at the end.

Algorithm: Identifying Leader Instructions

1. The **first statement** in the TAC program is a leader.
2. Any statement that is the **target of a conditional or unconditional goto** is a leader.
3. Any statement that **immediately follows a conditional or unconditional goto** is a leader.

2. Directed Acyclic Graph (DAG) for Basic Blocks

A DAG represents basic block expressions without redundancy:

- Leaves represent initial variable values or constants.
- Interior nodes represent operators.
- Nodes are annotated with variable labels currently holding the computed value.

Optimization Technique	Description & Realized Benefits
Common Subexpression Elimination (CSE)	Identifies identical subexpression computations whose operand values have not changed, eliminating redundant operations.
Constant Folding	Evaluates operations with constant operands at <i>compile-time</i> (e.g., replacing <code>`2 * 3.1415 * r`</code> with <code>`6.283 * r`</code>).
Constant Propagation	Replaces variable uses with known compile-time constants.
Dead Code Elimination	Deletes instructions that compute values never used on any execution path.
Loop-Invariant Code Motion (Hoisting)	Moves computations whose operands are constant throughout the loop to the loop pre-header.
Induction Variable Elimination & Strength Reduction	Replaces expensive multiplications (e.g., $i \times 4$) inside loops with incremental additions ($t = t + 4$).

3. Register Allocation via Graph Coloring (Chaitin's Algorithm)

Given k hardware registers, we construct the **Interference Graph** where nodes are variables/temporaries and edges represent live ranges that overlap. If the graph can be colored with k colors such that no two adjacent nodes share the same color, no

register spilling occurs.

4. Peephole Optimization

Examines a short sliding window (peephole) of target instructions to apply local transformations:

- **Redundant Load/Store Elimination:** `MOV R0, x` followed by MOV x, R0` → remove 2nd move.`
- **Unreachable Code Elimination:** Removing instructions immediately following unconditional jumps.
- **Flow-of-Control Optimizations:** ``goto L1 ... L1: goto L2` → replace with `goto L2`.`
- **Algebraic Simplifications:** Replacing ``x = x + 0` or `x = x * 1` with no-op; replacing `x = x * 2` with `x = x << 1`.`